

Change Detection in Angular

A Beginner-Friendly Guide with Examples



Sreeram Saikumar

Angular Developer

 /sreeramsaikumar

What is Change Detection?

Change Detection (CD) is the process Angular uses to keep the UI (view) in sync with the data (model). It checks for changes in your data and updates the view accordingly.

Example:

Let's say we have a simple app where a user's name is displayed.

```
<input [(ngModel)]="userName">  
<p>Hello, {{ userName }}!</p>
```

As you type in the input field, Angular will automatically update the **p** tag because it detects that the **userName** property has changed.

When the **userName** changes, Angular automatically detects it and updates the view.

 /sreeramsaikumar

How Does Angular Know When to Update?

Angular triggers CD based on:

1. User Interactions (e.g., typing, clicking).
2. Async Operations (e.g., HTTP requests).
3. Component Changes (e.g., changing input data).

 /sreeramsaikumar

Change Detection Strategies:

Angular provides two primary change detection strategies

1. Default (CheckAlways) Change Detection
2. OnPush Change Detection

 /sreeramsaikumar

1. Default Change Detection

By default, Angular runs change detection on **every component** whenever an event occurs.

Example:

```
@Component({
  selector: 'app-root',
  template: `
    <button (click)="updateData()">Update Data</button>
    <p>{{ data }}</p>
  `
})

export class AppComponent {
  data = "Old Data";

  updateData() {
    this.data = "New Data"; // Changes detected here
  }
}
```

Whenever you click the button, Angular checks for any changes in the **`data`** property and updates the view.

 /sreeramsaikumar

2. OnPush Change Detection

The `OnPush` strategy is more efficient. It only runs change detection if:

1. The input data changes (e.g., via `@Input()`).
2. An event occurs inside the component.

Example:

```
@Component({
  selector: 'app-child',
  template: `<p>{{ childData }}</p>`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ChildComponent {
  @Input() childData!: string;
}
```

Why Use OnPush?

- It **reduces unnecessary checks** and makes Angular apps more **performant**, especially in large applications.
- Only **specific components** are checked instead of the entire component tree.

 /sreeramsaikumar

Triggering Change Detection Manually

Sometimes, you need to trigger change detection manually, especially when dealing with third-party libraries or custom async code.

Example:

```
import { ChangeDetectorRef } from '@angular/core';

@Component({
  selector: 'app-manual-cd',
  template: `<p>{{ data }}</p>`
})
export class ManualCDComponent {
  data = "Initial Data";

  constructor(private cdRef: ChangeDetectorRef) {}

  updateData() {
    this.data = "Updated Data"; // Manual trigger needed
    this.cdRef.detectChanges(); // Manually trigger CD
  }
}
```

In this example, after updating `data`, we manually trigger CD using `this.cdRef.detectChanges()` to make sure the view gets updated

 /sreeramsaikumar

Zones and Async Operations

Angular uses **Zone.js** to track asynchronous operations like HTTP requests or `setTimeout`. When the operation completes, Angular triggers change detection automatically.

Example:

```
setTimeout(() => {  
  this.data = "Data from Timeout"; // Change detected  
}, 1000);
```

Even though `setTimeout` is asynchronous, Angular will detect the change after 1 second and update the view.

 /sreeramsaikumar

Change Detection with Observables

Angular integrates with **Observables** seamlessly. When an Observable emits a new value, Angular runs CD to update the view.

Example:

```
<div>{{ observableData | async }}</div>
```

Here, `observableData` is an **Observable**. When it emits new data, Angular automatically updates the view.

```
this.data$ = this.dataService.getData(); // This is an Observable
```

The **AsyncPipe** handles subscription and change detection automatically.

 /sreeramsaikumar

Performance Tips for Change Detection

- Use **`OnPush`** for components that don't change often.
- **Use immutable data:** When data is immutable (doesn't change in place), Angular can more efficiently detect changes.
- **Detach Change Detection:** You can detach a component from CD if it doesn't need to be checked often.

Example:

```
this.cdRef.detach(); // Detach component from change detection
```

 /sreeramsaikumar

Detach and Reattach Change Detection

You can also **detach** a component from Angular's change detection tree to improve performance.

Example:

```
this.cdRef.detach(); // Detach the component  
// Component won't be checked for changes unless you reattach it manually.
```

When necessary, you can **reattach** it:

```
this.cdRef.reattach(); // Reattach and trigger CD for this component
```

 /sreeramsaikumar

Key Takeaways:

- **Change Detection** ensures the view stays in sync with data.
- **Default** checks all components; **OnPush** is more efficient for large apps.
- Use **ChangeDetectorRef** to manually trigger change detection when needed.
- Angular uses **Zone.js** to track async operations and trigger CD automatically.
- **Performance optimization** is key with **OnPush** and immutable data.

 /sreeramsaikumar

Start Optimizing Your Angular Apps!

Now that you understand Change Detection, it's time to apply it and optimize your Angular apps!

Drop your questions in the comments and let's discuss how you can improve performance in Angular.

Don't forget to follow for more Angular things!

 /sreeramsaikumar

 /sreeramsaikumar

+ Follow

 /sreeramsaikumar